



Cal Poly
Pomona

College of Engineering

ECE 4301.01

Fall 2025

Dr. Mohammed E. Aly

Midterm 2 - TRNG on Raspberry Pi Crypto Engine

By Quinn Bell, Melvin Contreras, Aaron Tran

Random numbers are important in modern cryptography. They are used to make keys, nonces, and other values that must be hard to guess. If an attacker can predict these numbers, they may be able to break the whole system. There are two main ways to get random numbers. One way is to use a pseudorandom number generator (PRNG), which is an algorithm that takes a secret seed and produces a long sequence of bits. This is fast, but if someone knows or guesses the seed, they can reproduce the whole sequence. The other way is to use a true random number generator (TRNG). A TRNG uses unpredictable physical processes in the real world and turns those into random bits.

There are several common types of TRNGs. One big category is noise based TRNGs. At the electronic level, parts like resistors and diodes are never perfectly stable. In a resistor, electrons move randomly because of heat, causing small random voltage changes called thermal noise. A TRNG can amplify this noise and use a circuit to check whether the voltage at a certain moment is above or below a threshold. If it is above, it might output a “1”; if it is below, it might output a “0.” Over time, this gives a random stream of bits. A similar idea uses avalanche or Zener diodes. When these are reverse biased, they can produce irregular current pulses due to random physical events inside the device. When we amplify and digitize these pulses, we get random bits.

Another important category is timing based TRNGs. These use the fact that digital circuits do not switch at exactly the same time on every cycle. For example, a ring oscillator is a loop of inverters that produces a repeating signal. The exact timing of each edge changes slightly due to noise and other small effects; this is called jitter. A TRNG can sample one oscillator with another clock. Because of the jitter, the sampled value sometimes ends up as 0 and sometimes as 1 in a way that is hard to predict. Some designs also use metastability in flip flops, where the circuit is forced into a borderline condition and noise decides whether it settles to 0 or 1. In all of these designs, small physical variations in time are turned into random bits.

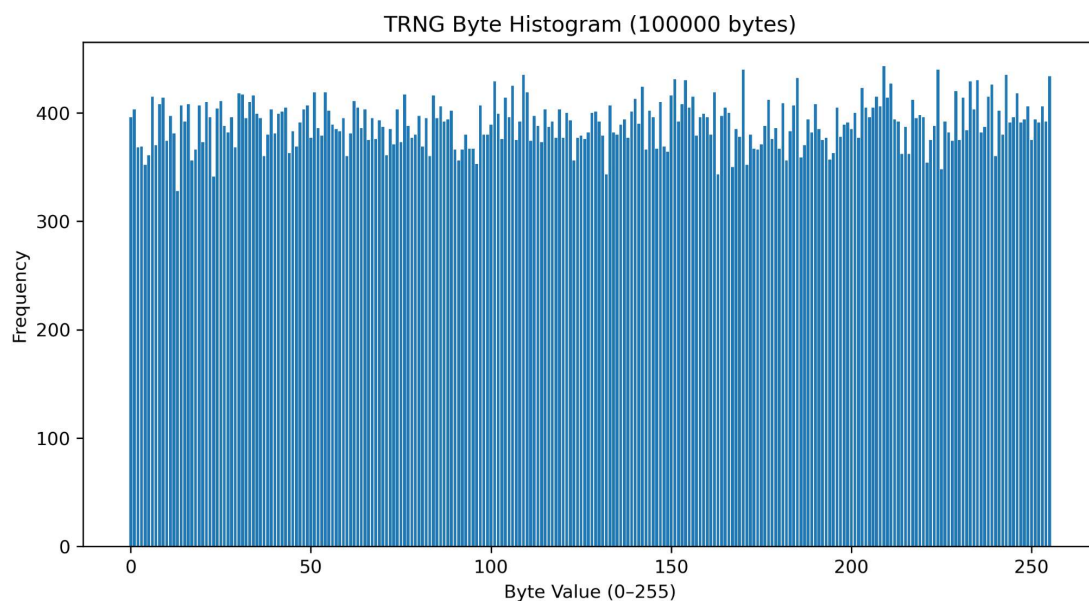
There are also quantum based TRNGs, which rely directly on quantum mechanics. For example, we can send single photons into a 50/50 beam splitter and record which detector they hit. Quantum theory says the outcome of each event is fundamentally random. These systems provide very high quality randomness but are more complex and expensive, so they are usually used in special high security systems rather than everyday devices. Some systems also use environmental sources, such as timing of key presses or network packets, as extra entropy, but these are often treated as a supplement to hardware noise sources.

Whatever physical source is used, the raw output of a TRNG usually is not perfect. There might be a slight bias (more 0s than 1s) or patterns in the data. To fix this, TRNGs use post-processing, also called conditioning. This can involve combining multiple samples or running the data through a cryptographic function to “smooth out” the distribution. TRNGs also run simple health tests to make sure they have not failed, such as checking that the generator is not stuck at all zeros.

In our project, we worked with the TRNG that is built into a hardware crypto engine and exposed in Linux as `/dev/hwrng`. We do not see the exact circuit inside, but based on similar

devices, it probably uses electronic noise or timing jitter plus conditioning and health checks. To see how good its output looks, our group wrote a Python program that reads 100,000 bytes from `/dev/hwrng` and performs some basic tests. We converted the bytes to bits and counted how many were 1 and how many were 0. In one run we got 800,000 bits total, with 400,623 ones and 399,377 zeros. This is very close to a 50/50 split, which is what we expect from a good random source.

We also counted how often each byte value (0 to 255) appeared. Each value showed up roughly a few hundred times, and when we plotted a histogram, the bars were fairly flat, with no obvious missing or overused values. These simple tests do not prove the TRNG is perfect, but they are good sanity checks. A badly broken generator would usually show strong bias or clearly uneven distributions even in a sample this size.



In conclusion, true random number generators take advantage of unpredictable physical effects, such as electronic noise, timing jitter, or quantum behavior to create random bits for cryptographic use. Our group's tests on the hardware TRNG accessed through `/dev/hwrng` showed an almost even split of 0s and 1s and an approximately uniform spread of byte values. Based on these results, the TRNG appears to be working properly and provides a reasonable entropy source for secure operations on our system.